What's new

A paper diagram visualizer

Terence Tao · 13 July 2026

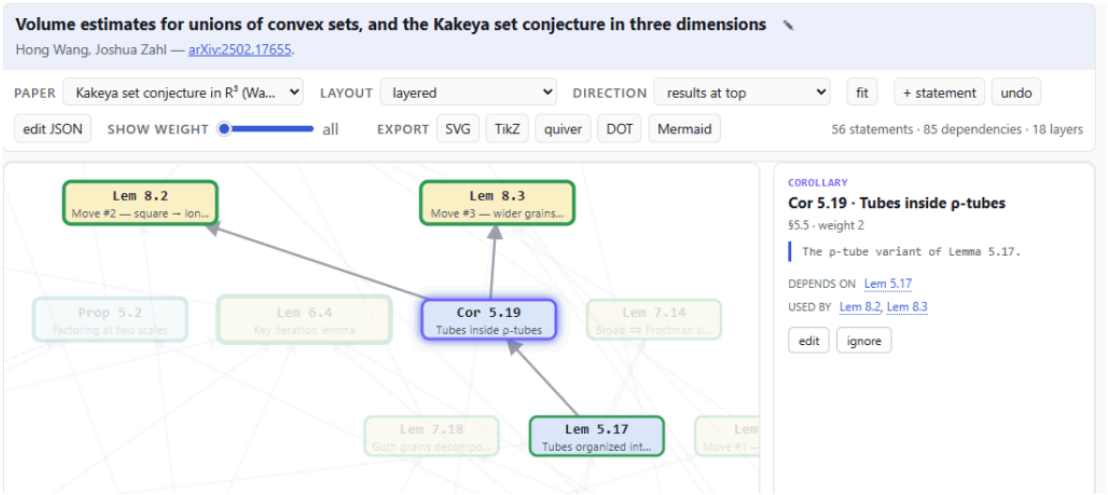
https://terrytao.wordpress.com/2026/07/13/a-paper-diagram-visualizer/

tao2typ · converted to Typst 2026-10-14 (UTC)

I am finding the newly revealed capability to code old applet ideas into reality to be very tempting to sink more time into, though I am certainly encountering the common "vibe coding" experience that the process can produce something that superficially resembles a finished product well before a satisfactory level of testing and review has been completed; indeed, it is the review process which is now the most time-consuming, to the point where I think any further advances in coding agent capability will have little impact on the new bottlenecks in the design process.

In any event, I spent a few hours working to realize a proposal I had made back in 2023 to automatically create diagrams to visually illustrate the logical flow of a given mathematical paper. At the time, Freddie Manners, extrapolating from the half-decent capability of the then-newly released ChatGPT 3.5 at this task, presciently predicted that "by the time a dedicated tool had been completed, the next general purpose engine would be better than it".

With that in mind, I decided to focus not on the generation of the diagram – which now can be done at various levels of quality by any number of large language models – but on its presentation. The result is the following app, which can take a certain formatted JSON file of dependencies between theorem objects and produce an interactive graph which can be explored, edited and also exported (somewhat lossily) into other standard formats such as SVG, TikZ, quiver, or Mermaid. Here is a screenshot of a diagramming of the celebrated proof by Wang and Zahl of the three-dimensional Kakeya conjecture:



Using an LLM, I generated diagrams for eight papers for demonstration purposes, including for instance a diagram for Wiles's proof of Fermat's last theorem, or of Szemeredi's proof of his famous theorem on arithmetic progressions (which sports a notoriously convoluted such diagram in the original paper), as well as a few papers of my own. If there are other requests to diagram particular papers, I can try to use an LLM to generate more examples; but my intention is for users of the app to create their own such diagrams, either by manually constructing them, or by directing their own AI tools to build the diagram in the required format (which is a JSON, with the precise specification given here).

I mentioned in the previous post that for these sorts of visualization apps, which work deterministically for a given set of inputs, the downside risk of LLM use to build the app is acceptably low. For this particular app, there is a complicating factor, which is that while the app does remain deterministic, the data I used to populate the app – namely, the above diagrams – are also LLM-generated. I have done spot

checks comparing the diagrams against the source papers, and did not find any errors; however, they are not guaranteed to be 100% accurate, and should only be used as approximations to the logical structure of these papers rather than completely exact representations. (The latter might become deterministically extractable should the results of these papers become formalized in a proof assistant language, but this is not currently the case.) Still, I hope these sorts of diagrams can serve as a helpful initial guide when first trying to read and understand a complex paper.

Comments (16)

Anonymous · 13 July 2026

You lay out three options: prose, LLM-generated diagram, and full formalization. There's a fourth option, the author just writes down the dependency structure while writing the proof. This is basically Lamport's idea from "How to Write a 21st Century Proof" (https://lamport.azurewebsites.net/pubs/proof.pdf). If you write a proof as a hierarchy of claims, each with its own hypotheses and goal, you've already written the dependency graph. Keep that in the markup instead of flattening it into prose, and the graph can be generated deterministically. It costs almost nothing extra, since you're just writing the proof carefully.

Once the structure is written down this way, you can do more than generate a structural diagram. You can collapse a section or proof, see what a step depends on and what depends on it, reorder steps that are independent, and click through the graph, all on the actual text. I've been working on an implementation of this, which I will soon submit to the AMR's Paper of the Future prize. In case a concrete example helps: https://github.com/leotrs/potf/. Video to come soon.



Terence Tao · 13 July 2026

This reminds me of the Lean Blueprint plugin for LaTeX, which has become a de facto standard for the first phase of a Lean formalization project in which one wants to see a structured informal proof of a given result before fully formalizing each of the components. There is a more recent Lean package, LeanArchitect, which embeds the blueprint directly into the Lean source and extracts as much of the metadata as it can automatically from the structure. In this model, the informal argument can be written through Lean stubs annotated with LeanArchitect LaTeX comments.

In short, the lines between formal proof, informal proof, and structural proof metadata are rapidly blurring, and I can certainly see all three being generated in tandem in the near future.



Mercury Stills · 20 July 2026

Professor Tao,

This observation—that the lines between formal proof, informal proof, and structural proof metadata are blurring—is strikingly close to something I have been building since December 2025.

Strategic.GG's Open Problem Gallery is a graph-based workspace for coordinating mathematics research. Inspired by the Polymath Project, it maps a complementary stage of the work: not the retrospective logical skeleton of a finished paper, but the live search that precedes one—claims, dependencies, strategy branches, checks, failed or disputed approaches, challenges, and forks. Lean formalizations can be attached to individual claims and independently reviewed.

OPG is now public here:
https://strategic.gg/open-problem-gallery/

I describe its coordination, verification, and falsification model here:
https://strategic.gg/s/748/

Your JSON grammar and the Blueprint converter you describe suggest an interoperability opportunity. I have not implemented that interchange layer; OPG currently uses its own graph representation. But I would be interested in whether a small common profile could let a live problem-solving graph mature into a paper-dependency diagram or Lean Blueprint while retaining provenance, fork lineage, and review history.

Disclosure: I'm Mercury, Strategic.GG's founder and developer.

Anonymous · 13 July 2026

Hi Terry, how cool!
Wonder if there could be a bridge to Lean, via a dependency diagram like this one: https://thefundamentaltheor3m.github.io/Sphere-Packing-Lean/blueprint/dep_graph_document.html

> **Terence Tao** · 13 July 2026
>
> Nice suggestion! With a little LLM prompting I was able to convert Viazovska's sphere packing blueprint into the format of my app: https://teorth.github.io/tao-web/apps/paper-diagram.html?paper=sphere-packing-lean . I am directing the agent now to build a converter tool that can deterministically convert any such blueprint into the JSON format for my app; it cannot be done inside the app itself because it requires web access, which I have disabled for my applets, but will be attached to the applet page as a separate downloadable piece of python code (wrapped around a small script that fetches the appropriate files from the web).

Anonymous · 13 July 2026

Some time ago I mentioned a more elaborated functionality. First I recommended using git@github.com:michal-h21/rdfref.git.
It is a flexible tool, though not automatic, and its syntax differs somewhat from standard LaTeX. Recently a Chinese software engineer, Cj Looi, launched a webpage that uses a JSON parser to generate dependency graphs automatically from LaTeX source files.
https://www.sciencestack.ai/upload
For the past two months I have been testing the software and suggesting features. It now seems reasonably mature. Disclaimer: I have no connection to Mr Looi, but I think his work is serious and worth mentioning. The paper you cited in your blog has since been uploaded, and the site can generate its dependency graph: A counterexample to the periodic tiling conjecture (there is a dark and light mode; the default is dark). A few observations: By default the software generates a graph using all labels and references from theorem, definition and similar environments. These can be deactivated if the graph becomes confusing. Blocks can be moved manually for better visibility, though the arrangement is only temporary. In your manuscript, page 5, you included your own dependency graph, which is smaller than the automatically generated one. On request, Mr Looi might add support for special labels, for example , so that only tagged environments appear. This feature is not yet implemented. Some papers from the ArXiv have been uploaded, but by no means all. You can create your own account to upload manuscripts. A free and a paid tier are available, though the latter seems premature for now. I hope you find this useful.
Uwe Brauer

CJ Looi · 13 July 2026

Agreed that this can be a very useful tool for mathematicians, the UI looks quite similar to what I've built here https://www.sciencestack.ai/graph/kakeya-restriction
It's a proof of concept and the data is also LLM-generated, but one could eventually annotate or extend the graph, or use LLMs to pull from existing literature to connect more papers

> **CJ Looi** · 13 July 2026
>
> I have open sourced some of the proof graphs in this repository https://github.com/sciencestack-ai/proof-dags

Elias Lam · 13 July 2026

This job is really great. When dealing with complex papers, I believe this tool can greatly improve efficiency.

Also, I found a post that somewhat overlaps with your idea. The difference is that it analyzes a textbook, and it looks a bit cluttered: https://onehappybird.wordpress.com/2012/12/03/whats-the-most-important-theorem/
But this might suggest that this approach could be used not only to analyze papers, but also various textbooks and lecture notes, right?

 **domotorp** · 13 July 2026

Feedback on the interface: for me zooming scrolls work terribly, even if I want to zoom out a little, it makes the whole diagram invisibly tiny.

>  **Terence Tao** · 13 July 2026
>
> Thanks for this. There was indeed a bug (invisible when testing on my device, but apparently possible on other devices, particularly ones involving a trackpad) where a scrolling gesture triggered multiple zoom out calls at once instead of just one. Hopefully it is fixed now (and also in the relativity app, which my agent discovered had the same bug); let me know if it still persists!

 **Anonymous** · 14 July 2026

Just for that 1% chance that you might like to contribute to the newly born bilingual Farsi–English journal People and Mathematics: your intentions and motivations behind creating these apps would fit beautifully in the journal's "Concepts and Conceptions" section:https://mathswell.com/people-and-mathematics/

 **Tim Ktitarev** · 14 July 2026

What a nice tool! I gave Gemini my bachelor's thesis as a single TeX file and instructed it to generate a JSON for a diagram. It took some convincing to generate all the relevant nodes and edges. I realized that often in the thesis I did not make an explicit reference to what I was using because from the context it was clear, but for Gemini it was not. You can see my chat and the end result here.
I did not thoroughly check if all the dependencies were right, but it looks like it. A few comments on the end result. My thesis was about a numerical scheme called the continuous Petrov-Galerkin method. The main theoretical task was understanding a proof of convergence from the literature. On the left side of the diagram, there are some introductory results. On the right side, there is the main bulk of the thesis. At the top is the main proposition on the convergence of the scheme. I reformulated the proof ideas from the literature into two other results – a "consistency" and "stability" result, which in combination given an alternative proof of convergence. You can very nicely see in the diagram how the definitions of the scheme, the definition of convergence as well as the consistency and stability go into the proposition of convergence.
I like it! I can also see myself using this to try to get a quick overview of a paper and see which parts are really relevant to my own work.

>  **Terence Tao** · 14 July 2026
>
> Great to see that the workflow of giving the JSON format to a third-party LLM works! I have uploaded your JSON (as the first reader-contributed example to the catalog) at https://teorth.github.io/tao-web/apps/paper-diagram.html?paper=cpg-ktitarev
> EDIT: I have now placed my apps under an MIT license, so that users are permitted to host local copies of this app in order to display their own curated libraries of paper diagrams distinct from the sample ones shown on this site, as long as the copyright and license is retained.

Pavel Ievlev · 19 July 2026

Hi! My wife recently wrote an app for this. Initially for my personal use, but now she plans to release it publicly. It's called proofmap, you can try it out here: https://proofmap.aleph.science/ . It can automatically parse papers in tex/json format into theorem blocks with dependencies. I'm already using it intensively, although it's still in the stage of active development. It has mcp/http api for using with claude code or other agents, although the workflow is not ai-oriented. Any feedback is welcome through the feedback button in the app :)

Anonymous · 25 July 2026

here are computer science style diagrams via code for inspiration https://cdn-0.plantuml.com/